Jacaranda Lexical Analyser

Team 1:
Aguilar Lara Ericka
Bautista Reyes Sofía
Cruz Basilio Ximena Carolina
Garibay Zamorano Josué Benjamín
Jardón Marín Rodrigo

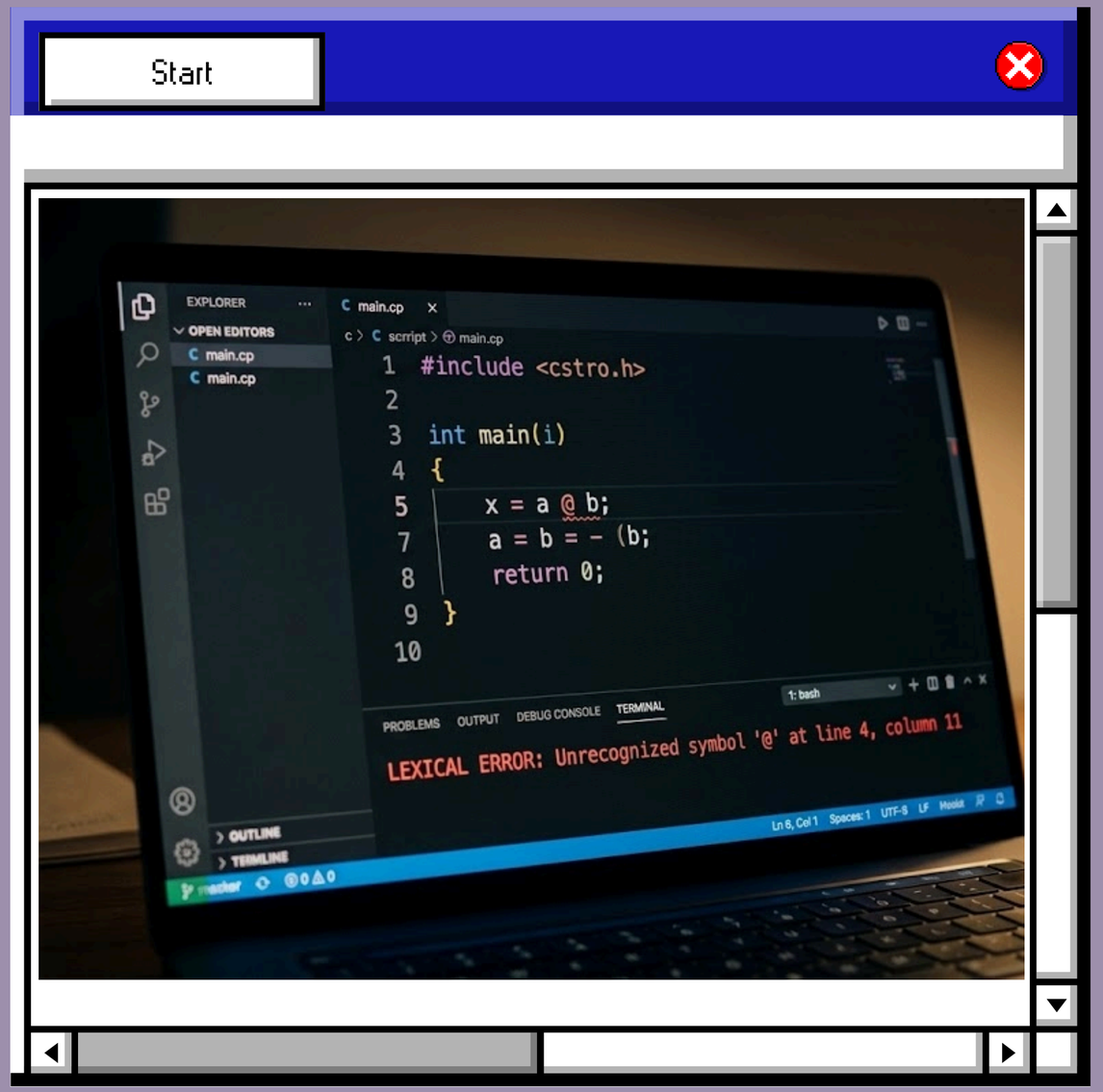
The first line of defense to your code

11:11PM

Messy code =
Wasted time.

The Uncaught syntax errors stop the production pipeline.







Source code

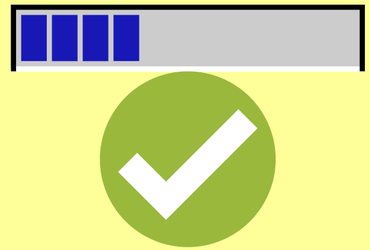
```
int a = 10;
int b = 20;
int c = a + b;
print(c);
if (c >= 20) {
    print("result");
}
print(a);
```

Analysis result

```
keyword      print
punctuation  (
identifier   c
punctuation  )
punctuation  ;
keyword      if
punctuation  (
identifier   c
operator     >=
constant     20
punctuation  )
punctuation  {
keyword      print
punctuation  (
constant     "result"
punctuation  )
punctuation  ;
punctuation  }
keyword      print
punctuation  (
identifier   a
punctuation  )
punctuation  ;

Token sequence:
keyword identifier operator constant punctuation keyword identifier operat

Total of tokens: 40
```







PURE FAST AND STANDARDIZED TOKENIZATION

Error Handling

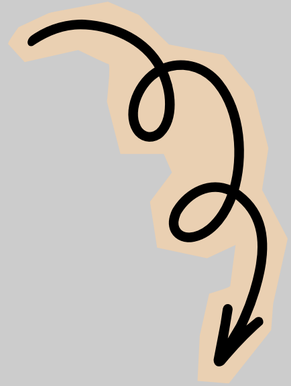
Surgical debugging



We don't crash. We pinpoint the exact failure

Source code

```
int a = 10 @ 3;
```



Analysis result

```
LEXICAL ERROR: unrecognized symbol '@' at line 1, column 12.

  int a = 10 @ 3;
             ^
```

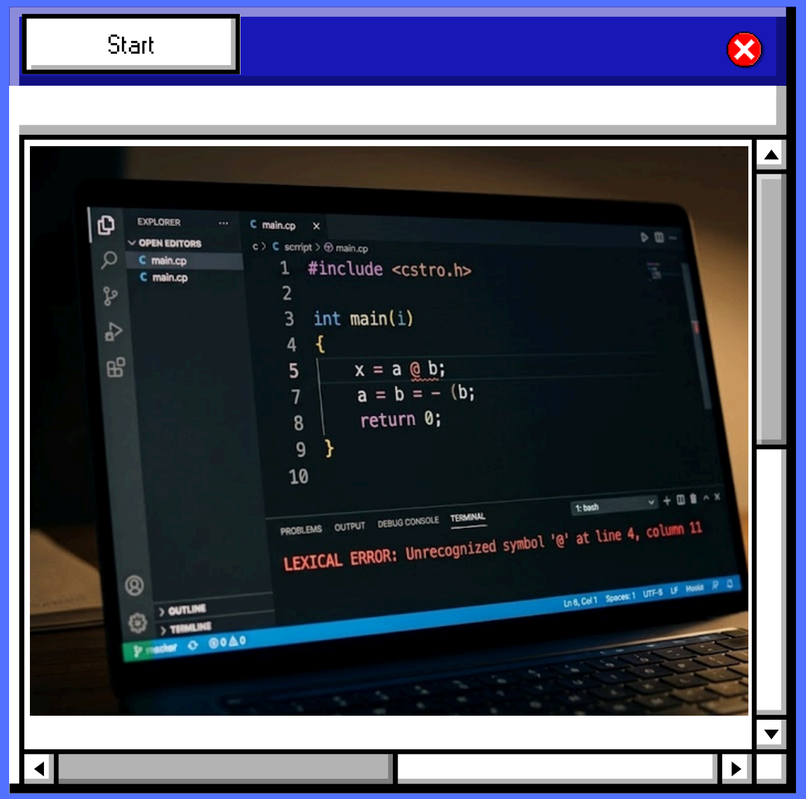




JACARANDA G5.01: The Complete Front-End

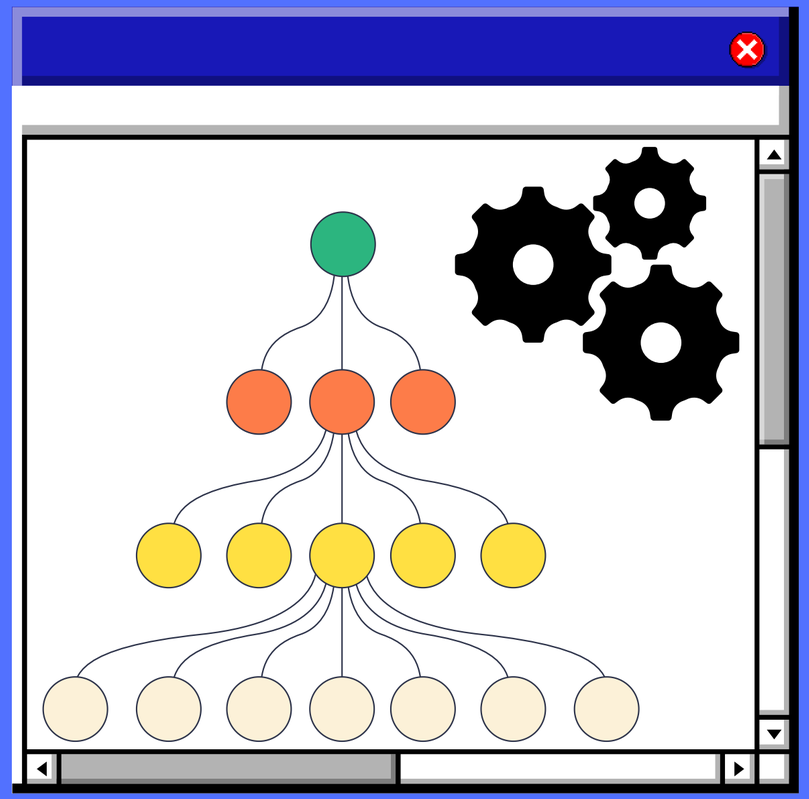
v1.0 - Lexical Analyzer

Phase 1: Words & Tokens

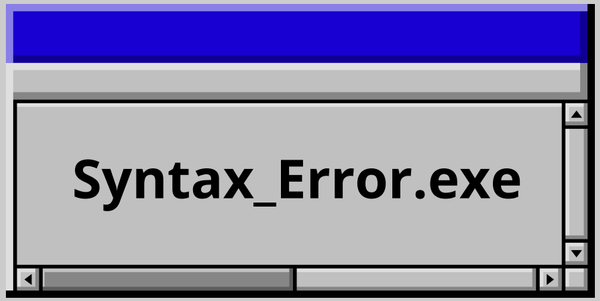


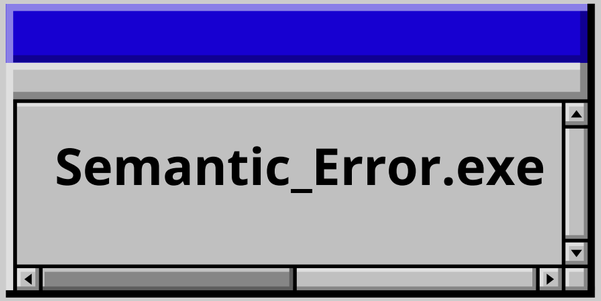
v2.0 - Parser & SDT

Phase 2 & 3: Structure & Logic



The Next Level: Syntax vs. Semantics





int x = ;

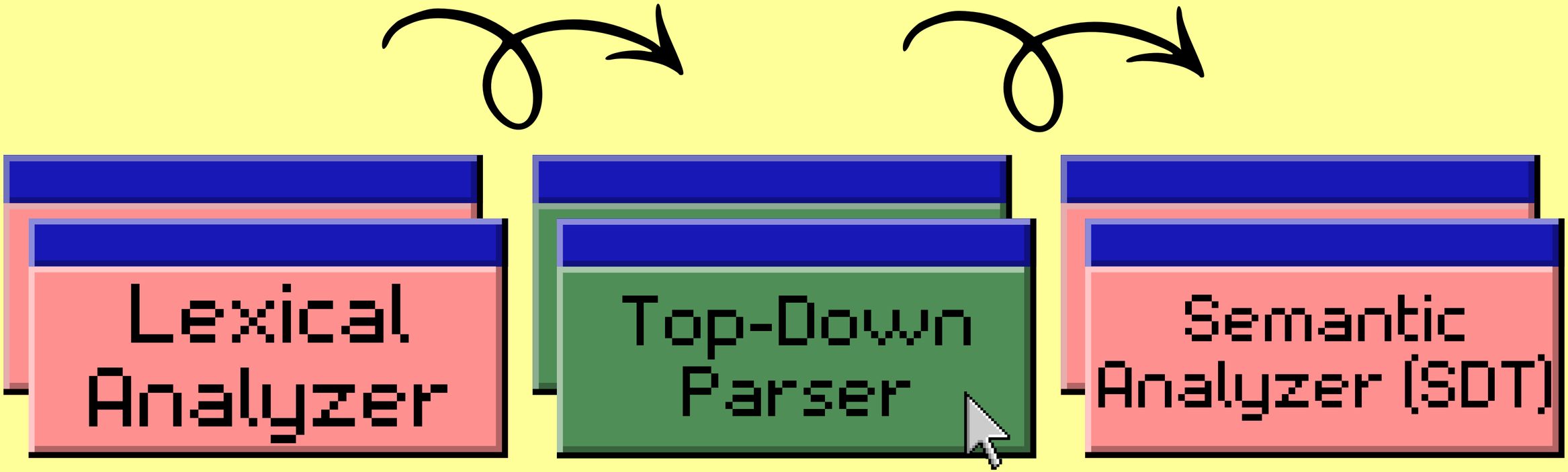
int x = true;

Parsing error: Expected an expression

SDT error: Type mismatch: cannot assign bool to int

Grammar broken (Incomplete instruction)

Grammar OK, Logic broken



RPLY Engine (Tokens)

100% Manual (No black-boxes). AST Generation.

Dynamic Symbol Table & Type Checking.

Scope & Capabilities



A C-like subset

Syntax Rules	Semantic Rules
Variables & Math (Strict Precedence)	Duplicate Declarations
Control Flow (if, else, while, for)	Undeclared Variables & Functions
Functions & Scoped Blocks	Type Compatibility (e.g., int to float)

Jacaranda - Lexer, Parser & SDT

JACARANDA

Lexer / Parser / SDT

Analyze | Parser & SDT | View Parse Tree | Open File | Clear

The Brain on Screen (Parse Tree & AST)

Parse Tree: Maps each grammatical component (including punctuation)

AST: Clean up the visual noise. Leave only the logical essence.

Source code

```
int x = 20;
```

Analysis result

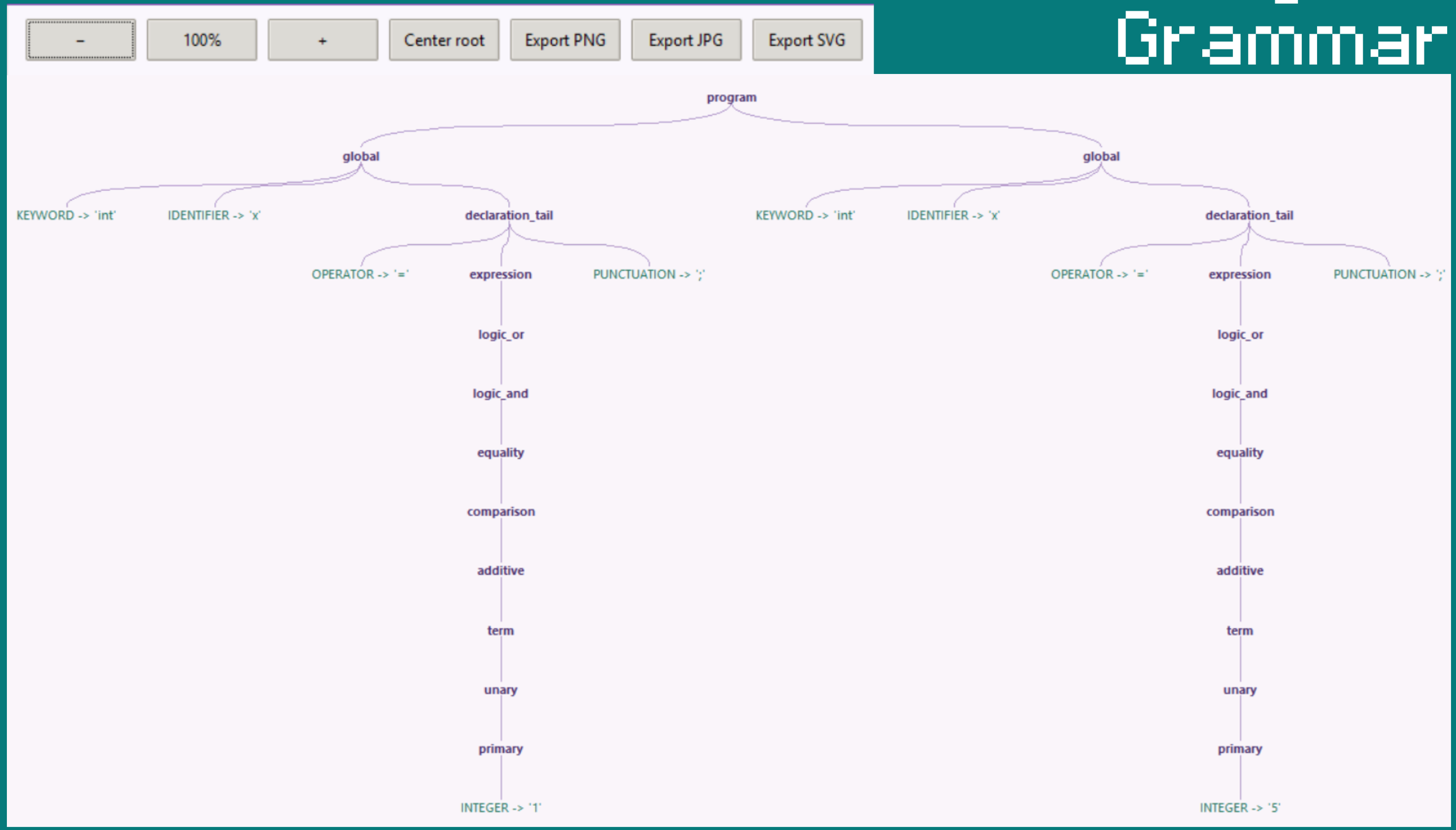
```
Parsing Success!

Parse tree:
program
  global
    KEYWORD -> 'int'
    IDENTIFIER -> 'x'
    declaration_tail
      OPERATOR -> '='
      expression
        logic_or
          logic_and
            equality
              comparison
                additive
                  term
                    unary
                      primary
                        INTEGER -> '20'
    PUNCTUATION -> ';'
  EOF

Abstract syntax tree:
Program
  Declaration (int x)
    Literal (INTEGER:20)

SDT Verified!
```

Visualizing the Grammar



Surgical Debugging in Action

Source code

```
int x = 20;
```

Analysis result

```
Parsing Success!

Parse tree:
program
  global
    KEYWORD -> 'int'
    IDENTIFIER -> 'x'
    declaration_tail
      OPERATOR -> '='
      expression
        logic_or
          logic_and
            equality
              comparison
                additive
                  term
                    unary
                      primary
                        INTEGER -> '20'
      PUNCTUATION -> ';'
  EOF

Abstract syntax tree:
Program
  Declaration (int x)
    Literal (INTEGER:20)

SDT Verified!
```

Perfect Case



Source code

```
int x = true;
```

Analysis result

```
Parsing Success!

Parse tree:
program
  global
    KEYWORD -> 'int'
    IDENTIFIER -> 'x'
    declaration_tail
      OPERATOR -> '='
      expression
        logic_or
          logic_and
            equality
              comparison
                additive
                  term
                    unary
                      primary
                        BOOLEAN -> 'true'
      PUNCTUATION -> ';'
  EOF

Abstract syntax tree:
Program
  Declaration (int x)
    Literal (BOOLEAN:true)

SDT error in line 1: Type mismatch: cannot assign bool to int
```

Syntax passes, semantics fails



Surgical Debugging in Action



Source code

```
int x = 1;
int x = 5;
```

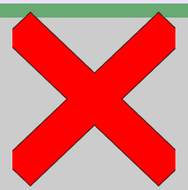
Analysis result

```
                        INTEGER -> '1'
      PUNCTUATION -> ';'
  global
    KEYWORD -> 'int'
    IDENTIFIER -> 'x'
    declaration_tail
      OPERATOR -> '='
      expression
        logic_or
          logic_and
            equality
              comparison
                additive
                  term
                    unary
                      primary
                        INTEGER -> '5'
      PUNCTUATION -> ';'
  EOF

Abstract syntax tree:
Program
  Declaration (int x)
    Literal (INTEGER:1)
  Declaration (int x)
    Literal (INTEGER:5)

SDT error in line 2: Variable 'x' is already declared
```

Scope Collision /
Duplicate Declaration



Jacaranda G5.01 - License Setup



100,000 tokens/month

Developer (FREE)

Lexical Analysis Only

Unlimited tokens

Pro ($35 per Month)

Full Front-End: Parser + SDT

